



DOCKER-CONTAINER- MANAGEMENT

Prepared by
Avishek Paudel



avishekspaudel12@gmail.com

Table of Contents

Introduction.....	3
Install Docker	3
Start Docker Service.....	4
Search Image	4
Pull Image	5
Run Container	5
Check Running Containers	6
Test Container.....	6
Execute Command Inside Container	7
Run Second Container.....	7
Stop / Start / Remove	8
Hello World Test.....	8

Introduction

This report documents the steps performed during Docker workshop 5. The objective of this workshop was to install Docker, run containers, manage them, and verify functionality through practical commands. Screenshots are included to provide evidence of successful execution.

Install Docker

Command: `sudo pacman -Syy docker`

This installs Docker using Arch Linux's package manager

```
We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

For security reasons, the password you type will not be visible.

[sudo] password for avishek:
:: Synchronizing package databases...
   core                               124.3 KiB   310 KiB/s  00:00 [#####] 100%
  extra                               8.2 MiB   3.90 MiB/s  00:02 [#####] 100%
resolving dependencies...
looking for conflicting packages...

Packages (4) containerd-2.2.2-1 libtool-2.6.0-4 runc-1.4.2-1 docker-1:29.4.0-1

Total Download Size:   51.56 MiB
Total Installed Size: 217.43 MiB

:: Proceed with installation? [Y/n] y
:: Retrieving packages...
 libtool-2.6.0-4-x86_64             426.8 KiB   453 KiB/s  00:01 [#####] 100%
 runc-1.4.2-1-x86_64                 3.1 MiB   1692 KiB/s  00:02 [#####] 100%
 containerd-2.2.2-1-x86_64          28.8 MiB   1029 KiB/s  00:21 [#####] 100%
 docker-1:29.4.0-1-x86_64           27.2 MiB   1165 KiB/s  00:24 [#####] 100%
Total (4/4)                         51.6 MiB   2.15 MiB/s  00:24 [#####] 100%
(4/4) checking keys in keyring [#####] 100%
(4/4) checking package integrity [#####] 100%
(4/4) loading package files [#####] 100%
(4/4) checking for file conflicts [#####] 100%
(4/4) checking available disk space [#####] 100%
:: Processing package changes...
(1/4) installing libtool [#####] 100%
(2/4) installing runc [#####] 100%
Optional dependencies for runc
  criu: checkpoint support
(3/4) installing containerd [#####] 100%
(4/4) installing docker [#####] 100%
Optional dependencies for docker
  btrfs-progs: btrfs backend support
  pigz: parallel gzip compressor support
  docker-buildx: extended build capabilities
:: Running post-transaction hooks...
(1/3) Creating system user accounts...
Creating group 'docker' with GID 960.
(2/3) Reloading system manager configuration...
(3/3) Arming ConditionNeedsUpdate...
[avishek@archlinux ~]$
```

Installed.

Start Docker Service

Command:

`sudo systemctl enable docker.service` = enables Docker at boot.

`sudo systemctl start docker.service` = starts Docker immediately.

Docker runs as a background service called the Docker daemon.

```
[avishek@archlinux ~]$ sudo systemctl enable docker.service
[avishek@archlinux ~]$ sudo systemctl start docker.service
[avishek@archlinux ~]$
```

Search Image

Command: `docker search nginx`

- Searches Docker Hub for available nginx images.

```
[root@archlinux avishek]# docker search nginx
NAME                DESCRIPTION                STARS     OFFICIAL
nginx               Official build of Nginx.   21252     [OK]
nginx/nginx-ingress NGINX and NGINX Plus Ingress Controllers fo. 118
nginx/nginx-prometheus-exporter NGINX Prometheus Exporter for NGINX and NGIN. 51
nginx/nginx-ingress-operator NGINX Ingress Operator for NGINX and NGINX P. 3
nginx/unit          This repository is retired, use the Docker o. 66
nginx/nginx-quic-qns NGINX QUIC interop        1
nginx/nginxaas-loadbalancer-kubernetes          1
nginx/unit-preview  Unit preview features     0
bitnami/nginx       Bitnami Secure Image for nginx 203
bitnamicharts/nginx Bitnami Helm chart for NGINX Open Source      3
ubuntu/nginx        Nginx, a high-performance reverse proxy & we. 141
kasmweb/nginx        An Nginx image based off nginx:alpine and in.  8
rancher/nginx        3
linuxserver/nginx    An Nginx container, brought to you by LinuxS. 235
dtagdevsec/nginx     T-Pot Nginx                0
paketobuildpacks/nginx 5
chainguard/nginx      Build, ship and run secure software with Cha. 5
vmware/nginx          3
gluufederation/nginx  A customized NGINX image containing a consu. 1
antrea/nginx          Nginx server used for Antrea e2e testing     0
cleanstart/nginx      Secure by Design, Built for Speed, Hardened . 0
activestate/nginx     ActiveState's customizable, low-to-no vulner. 0
intel/nginx           0
docksal/nginx         Nginx service image for Docksal              1
geokrety/nginx        Our customized nginx image                   0
[root@archlinux avishek]# _
```

Pull Image

Command: `docker pull nginx`

- Downloads the nginx image locally.

```
[root@archlinux avishek]# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
448ea5cac5d5: Pull complete
5435b2dcdf5c: Pull complete
054715a6bffa: Pull complete
88d1d984b765: Pull complete
4a038fd18db1: Pull complete
84e114c2bb36: Pull complete
7b5d674621c2: Pull complete
3eff0a97d435: Download complete
cc0cf959117b: Download complete
Digest: sha256:7f0adca1fc6c29c8dc49a2e90037a10ba20dc266baaed0988e9fb4d0d8b85ba0
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
[root@archlinux avishek]# _
```

Run Container

Command: `docker run --name my-nginx -d -p 8080:80 nginx`

- Creates and runs a container named my-nginx.

- -d runs in detached mode

- -p 8080:80 maps host port 8080 to container port 80.

```
[root@archlinux avishek]# docker run --name my-nginx -d -p 8080:80 nginx
2b75c38fb9ddda772d96c73ea5b8a30fa57d27bcbe822440b04c4cfa1fa27b83
[root@archlinux avishek]# _
```

Check Running Containers

Command: `docker ps`

- lists active containers with details like ID, name, ports and status

```
root@archlinux avishek1# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
2b75c30fb9dd   nginx    "/docker-entrypoint.."   About a minute ago   Up About a minute   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   my-nginx
root@archlinux avishek1# _
```

Test Container

Command: `curl http://127.0.0.1:8080`

- Confirms nginx is serving content via port mapping.

```
root@archlinux avishek1# curl http:// 127.0.0.1:8080
curl: (3) URL rejected: No host part in the URL
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@archlinux avishek1# _
```

Execute Command Inside Container

Command: `docker exec my-nginx cat`

`/usr/share/nginx/html/index.html`

- Display the default nginx HTML file inside the container.

```
[root@archlinux avishek]# docker exec my-nginx cat /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
[root@archlinux avishek]#
```

Run Second Container

Command: `docker run --name my-nginx2 -d -p 8081:80 nginx`

- Demonstrate running multiple containers simultaneously.

```
[root@archlinux avishek]# docker run --name my-nginx2 -d -p 8081:80 nginx
6bdce26369a172b78a73035af2a21da8510212f44db40d63161683f2d30507f8
[root@archlinux avishek]#
```

Stop / Start / Remove

Commands:

docker stop my-nginx = stops the containers

docker start my-nginx = restarts it

docker rm my-nginx2 = removes the second container

```
[root@archlinux avishek]# docker stop my-nginx
my-nginx
[root@archlinux avishek]# docker start my-nginx
my-nginx
[root@archlinux avishek]# docker stop my-nginx2
my-nginx2
[root@archlinux avishek]# docker rm my-nginx2
my-nginx2
[root@archlinux avishek]# _
```

Hello World Test

Commands:

1. docker pull hello-world
2. docker run hello-world

```
[root@archlinux avishek]# docker pull hello-world
Using default tag: latest
dockerlatest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest
docker.io/library/hello-world:latest
[root@archlinux avishek]# docker run hello-world
```

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
\$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
<https://hub.docker.com/>

For more examples and ideas, visit:
<https://docs.docker.com/get-started/>

```
[root@archlinux avishek]# _
```


